

Chapter 06: Legacy Spaghetti to Clean Design Patterns Refactoring

The Reality of Brownfield Architecture Many senior LLD interviews no longer ask you to design a Parking Lot from scratch. Instead, the interviewer presents a 400-line monolithic "God Class" that currently handles an e-commerce checkout flow and says:

"Our company grew 10x. This class has become impossible to maintain. Every new feature breaks something else. Refactor this code into an extensible, enterprise-grade architecture using appropriate design patterns."

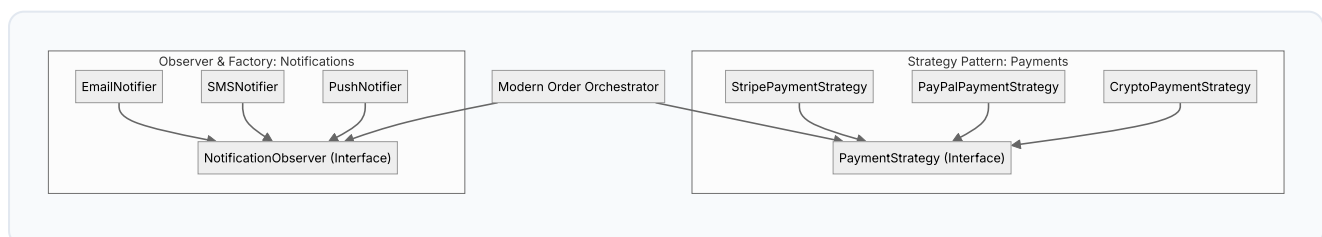
This chapter walks through the surgical decomposition of a messy real-world monolith into clean, testable Design Patterns.

1. The Monolithic God-Class

Here is the unrefactored codebase. Note how it violates every single SOLID principle:

1. **Single Responsibility Violation:** It calculates prices, processes credit cards, sends emails, sends SMS, and updates database tables.
2. **Open/Closed Violation:** Adding Apple Pay requires modifying 4 different `if-elif` blocks inside the class.
3. **Dependency Inversion Violation:** It directly instantiates concrete third-party SDK clients (`StripeClient()`, `TwilioClient()`).

2. The Refactoring Blueprint: Applying Design Patterns



We map each code smell to its standard architectural remedy:

- **The Payment `if-elif` chain** → **Strategy Pattern**: Encapsulate each payment gateway behind a uniform `PaymentStrategy` interface.
- **The Notification dispatch** → **Observer Pattern**: When an order succeeds, publish an `OrderPaidEvent`. Listeners handle SMS, Email, and Push asynchronously without the checkout service caring.
- **The Pricing & Discounts calculation** → **Chain of Responsibility / Decorator Pattern**: Separate pricing adjustments into composable rules.

3. The Clean Refactored Architecture

4. Verification and Extensibility Proof

Notice how easily we can now add a new payment provider (e.g., Apple Pay) or a new event hook (e.g., Data Warehouse Kafka Publisher) **without changing a single line of `ModernCheckoutService`**:

Further Reading

- [Refactoring catalog](#)
 - [Refactoring Guru: code smells](#)
 - [Strangler Fig pattern](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▼ 1. What is the first step when refactoring tangled code toward patterns?

Add tests that pin current behaviour, then make small behaviour-preserving steps.

- ▼ 2. Which smell often suggests Strategy?

Long `if/elif` chains selecting an algorithm by type or flag.

▼ 3. Which smell suggests Observer or events?

One class explicitly calling many others after a change.

▼ 4. Why refactor in small commits?

Each step is reversible and reviewable; failures are easy to localise.